

Introduction to JDBC

“How Java Talks to Databases —
From Theory to Interview Answers”



ADVANCED JAVA

Why Do We Need JDBC?

Applications need to **store, retrieve, and manage** data.

Java alone cannot do that — it needs a **bridge** between Java code and database.

This bridge is **JDBC (Java Database Connectivity)**.

JDBC = Standard API to connect Java applications with relational databases (MySQL, Oracle, PostgreSQL, etc.)

What Is JDBC?

JDBC is a Java API provided by Oracle that allows Java programs to :

- ✓ Connect to a database
- ✓ Send SQL queries
- ✓ Retrieve & process results
- ✓ Perform CRUD operations
- ✓ Handle transactions

JDBC is part of **Java Standard Edition** (JSE).

JDBC Architecture

(INTERVIEW-FAVORITE)

JDBC has two layers :

1. JDBC API (Java Layer)

Interfaces like:

- Connection
- Statement
- PreparedStatement
- CallableStatement
- ResultSet

2. JDBC Driver (Vendor Layer)

Database vendors provide the actual driver to communicate with DB.

E.g.,

- MySQL → `com.mysql.cj.jdbc.Driver`
- Oracle → `oracle.jdbc.driver.OracleDriver`

Interview Scoring Line:

“JDBC provides abstraction; drivers provide implementation.”

Types of JDBC Drivers

Type 1 – JDBC–ODBC Bridge Driver

 Deprecated. Slow. Not used anymore.

Type 2 – Native API Driver

Uses native DB libraries. Platform dependent.

Type 3 – Network Protocol Driver

Middleware converts requests. Rarely used.

Type 4 – Thin Driver (Most Used)

- ✓ Pure Java driver
- ✓ Platform independent
- ✓ Best performance
- ✓ Used by MySQL, Oracle

Always answer : “Modern apps use Type 4 drivers.”

Steps to Connect Java with Database

1. Load the Driver

```
Class.forName("com.mysql.cj.jdbc.Driver");
```

2. Establish Connection

```
Connection con = DriverManager.getConnection(  
    "jdbc:mysql://localhost:3306/mydb", "root",  
    "password");
```

3. Create Statement

```
PreparedStatement ps =  
con.prepareStatement("select * from student");
```

4. Execute Query

```
ResultSet rs = ps.executeQuery();
```

5. Process Result

```
while(rs.next()) {  
    System.out.println(rs.getString(1));  
}
```

6. Close Resources

```
con.close();
```

Interview Tip : Always mention resource cleanup.

Statement vs PreparedStatement

Feature	Statement	PreparedStatement
SQL Compilation	Every execution	Only once
Performance	Lower	Higher
SQL Injection Protection	 No	 Yes
Parameter Support	 No	 Yes (? placeholders)

Example :

```
PreparedStatement ps =  
con.prepareStatement(  
    "insert into student values (?, ?)");  
ps.setInt(1, 101);  
ps.setString(2, "Pratiksha");
```

Interview answer :

“PreparedStatement is preferred because it prevents SQL injection and improves performance.”

ResultSet

(Interview-Focused)

ResultSet is a table-like structure containing data returned from the database.

Useful methods :

- `next()` → move cursor
- `getInt()`, `getString()` → read columns
- `absolute()`, `previous()` for scrollable ResultSets

Types of ResultSet :

1. `TYPE_FORWARD_ONLY`
2. `TYPE_SCROLL_INSENSITIVE`
3. `TYPE_SCROLL_SENSITIVE`

Mentioning **ResultSet** types impresses interviewer immediately.

Transaction Management

Transactions = group of SQL statements executed as a single unit.

```
con.setAutoCommit(false);
```

```
ps1.executeUpdate();
```

```
ps2.executeUpdate();
```

```
con.commit();
```

If error :

```
con.rollback();
```

Uses :

- ✓ Money transfer
- ✓ Banking
- ✓ E-commerce orders

Interview line :

“Transactions ensure data consistency using commit and rollback.”

Common Interview Questions

Q1: Why use PreparedStatement over Statement?

- ✓ Prevents SQL injection
- ✓ Compiled once
- ✓ Faster and safer

Q2: Which JDBC driver type is used today?

- ✓ Type 4 (Pure Java driver)

Q3: What is the role of DriverManager?

- ✓ Establishes connection with DB using driver.

Q4: Difference between executeQuery() and executeUpdate()?

- executeQuery → SELECT
- executeUpdate → INSERT/UPDATE/DELETE

Q5: What is SQL injection?

- ✓ A vulnerability where attacker injects malicious SQL.
- ✓ Prevented by PreparedStatement.

Summary

- JDBC connects Java ↔ Database
- JDBC Architecture = API + Driver
- Type 4 driver is industry standard
- Steps: Load → Connect → Statement → Execute → Close
- PreparedStatement for secure + fast SQL
- ResultSet stores returned data
- Transactions ensure consistency

“JDBC is the backbone of Java backend — master it before moving to frameworks.”